

AI Code Review Report

File Name: Student2.py

This code snippet effectively demonstrates the concept of polymorphism in object-oriented programming. It's a clear and simple example.

1. Overall Code Quality Score: 7/10

The code is very simple, readable, and correctly illustrates polymorphism. It functions exactly as intended for its small scope. The main deductions are due to it being a bare-bones example lacking real-world features like data, constructors, or more robust naming conventions, which are expected in production-level code.

2. Bugs: None

There are no functional bugs in this code. It runs without errors and produces the expected output. It successfully demonstrates the polymorphic behavior of the `show` function.

3. Suggestions for Improvement:

1. **Meaningful Class Naming:** The class `Student2` is unusual. If it's the primary student class, it should simply be `Student`. If there's a reason for `2`, it should be clarified (e.g., `StudentV2`, `HighSchoolStudent`, etc.).

2. **Add `\_\_init\_\_` Constructors:** For real-world objects, classes should typically have an `\_\_init\_\_` method to initialize attributes. This would allow `display` to show actual data.

```python

class Student:

def \_\_init\_\_(self, name, student\_id):

self.name = name

self.student\_id = student\_id

def display(self):

print(f"Student Details: Name - {self.name}, ID - {self.student\_id}")

class Teacher:

def \_\_init\_\_(self, name, subject):

self.name = name

self.subject = subject

def display(self):

print(f"Teacher Details: Name - {self.name}, Subject - {self.subject}")

# Usage:

s = Student("Alice", "S123")

t = Teacher("Mr. Smith", "Math")

```

3. **Utilize Attributes in `display`:** Once attributes are added (via `\_\_init\_\_`), the `display` methods should use these attributes to provide more meaningful output.

4. **Docstrings:** Add docstrings to classes and methods to explain their purpose, especially in larger codebases.

```python

class Student:

```
"""Represents a student in the system."""
```

```
...
```

```
def display(self):
```

```
 """Prints the student's details."""
```

```
...
```

```
...
```

5. **Type Hinting:** Add type hints to the `show` function parameter for better readability and static analysis.

```
```python
```

```
from typing import Protocol
```

```
class Displayable(Protocol):
```

```
    def display(self):
```

```
    ...
```

```
def show(obj: Displayable):
```

```
    """Displays details of an object that has a 'display' method."""
```

```
    obj.display()
```

```
```
```

6. **Return vs. Print in `display`:** For more flexibility, `display` methods often *return* a formatted string rather than directly printing it. This allows the caller to decide whether to print, log, or store the information.

```
```python
```

```
class Student:
```

```
    # ...
```

```
    def display(self) -> str:
```

```
        return f"Student Details: Name - {self.name}, ID - {self.student_id}"
```

```
# Usage:
```

```
# print(s.display()) or print(show(s)) if show prints
```

```
def show(obj: Displayable):
```

```
    print(obj.display()) # Assuming display returns a string
```

```
```
```

7. **Main Guard:** For a script that might be imported elsewhere, wrapping the execution logic in `if __name__ == "__main__":` is a good practice.

```
```python
```

```
# ... (class and function definitions)
```

```
if __name__ == "__main__":
```

```
    s = Student2() # or Student("Alice", "S123")
```

```
    t = Teacher() # or Teacher("Mr. Smith", "Math")
```

```
    show(s)
```

```
    show(t)
```

```
```
```

```

```

#### ### 4. Best Practices:

1. **Polymorphism:** The code perfectly demonstrates polymorphism, where the `show` function can operate on objects of different types (`Student2`, `Teacher`) as long as they adhere to a common interface (having a `display` method). This is a core OOP principle.

2. **PEP 8 Naming Conventions:**

\* Class names use `CamelCase` (`Student2`, `Teacher`), which is followed.

\* Function names use `snake_case` (`show`), which is followed.

3. **Clarity and Simplicity:** For a demonstration of a concept, the code is very clear and concise,

which is excellent.

---

#### ### 5. Time Complexity:

\* \*\*`display` method:\*\* Each `display` method (for both `Student2` and `Teacher`) performs a single `print` operation. This is a constant-time operation.

\* Time Complexity:  $O(1)$

\* \*\*`show` function:\*\* The `show` function takes an object and calls its `display` method. Since the `display` method itself is  $O(1)$ , the `show` function is also a constant-time operation.

\* Time Complexity:  $O(1)$

\* \*\*Overall script:\*\* The script defines classes and a function, instantiates two objects, and calls the `show` function twice. All these operations (class/function definition, object instantiation, method calls) are constant time for this minimal example.

\* Overall Time Complexity:  $O(1)$  (constant time). The execution time does not grow with any input size, as there isn't any variable input.